UNIVERSITÀ DI PISA

DIPARTIMENTO DI INFORMATICA

Report
Assignment 3

Student:

Stefano Pea

ANNO ACCADEMICO 2024/2025

1 Execution Instructions

In order to test the programs, simply run the `script_sbatch.sh` script using the `sbatch` command inside the `spmcluster` machine.

```
sbatch script_sbatch.sh
```

This command will run `script.sh`, that will prepare the file to be compressed, using a list of commands similar to:

```
dd if=/dev/urandom of=bigfile.dat bs=2M count=50
```

Then it will compile the two source files for the provided sequential implementation `minizSeq.cpp` and the parallel one `minizPar.cpp`. The script will run a series of tests that we will discuss in the Test results section of this report.

2 Design and Implementation

The main structure of our program is based on the original implementation of `minizSeq.cpp`. The core of the program is the function `doWorkPar`, that chooses between four sub-functions based on user's request(compress/decompress) and file size.

Small files (size $<$ `CHUNK_SIZE`) invoke the sequential `compressSeq/decompressSeq` functions; larger ones invoke our chunked, parallel `compressPar/decompressPar` implementations.

On decompression, we inspect the header flag to call the matching routine (parallel or sequential encoded).

We've reused and lightly adapted much of the sequential code, including argument parsing, and simply extended `parseCommandLine()` with a new `-B` option to let users override the chunk size at runtime.

2.1 Header

The first byte of the file (`flag`) determines which decompression routine to invoke:

- `flag = 0` → use the sequential decompressor (`decompressSeq`).
- `flag = 1` → use the parallel (chunked) decompressor (`decompressPar`).

Sequential header format:

$$[\texttt{flag} \mid \texttt{origSize}] + \text{compressed data}$$

Parallel (chunked) header format:

$$[\texttt{flag} \mid \texttt{origSize} \mid \texttt{chunkCount} \mid \texttt{chunkSizes}] + \text{chunkData}$$

2.2 Parallelization

We parallelized the compressor with OpenMP using two nested loops:

- **Outer loop:** iterates over all input files (compress or decompress).
- **Inner loop:** processes chunks of a file when its size exceeds `CHUNK_SIZE`.

In our implementation, both loops use *static scheduling*, based on the following reasoning about the files to be compressed.

We will discuss the datasets in more details in Test results section, but the main idea is that all the files are generated randomly, so it should be expected the same amount of time to compress files (or chunks) of the same dimension, so it is not necessary to try to balance the workload among threads.

For real-world workloads with variable file or chunk sizes, one might prefer *dynamic scheduling* at one or both levels to balance unpredictable tasks.

We also configured the OpenMP runtime as follows:

- `omp_set_dynamic(0)` Ensures OpenMP spawns exactly the number of threads requested (no runtime downsizing).
- `omp_set_nested(1)` Enables nested parallel regions so that the inner chunk–loop can execute in parallel within the outer file–loop.

The total number of threads is specified at run-time via the environment variable `OMP_NUM_THREADS`.

3 Tests & Performance Analysis

Following the assignment requirements, we focused our experiments on compression performance and measured *strong speedup* relative to the provided sequential implementation. Two test scenarios were used:

1. **Large-file test:** A single 100 MiB file (`bigfile.dat`) was compressed to evaluate chunk-based parallelism. We varied
   - Number of OpenMP threads: `OMP_NUM_THREADS` = 1, 2, 4, 8, 16, 32
   - Chunk size: `-B` = 1, 2, 4, 8, 16 MiB
2. **Small-file test:** One hundred 10 KiB files were compressed to test file-level workload distribution. Here the chunk size was fixed at an arbitrary 6 MiB (exceeding each file's size) so that each file followed the sequential path, and only the thread count was varied.

For both scenarios, we used *static* scheduling on the outer (file) and inner (chunk) loops to eliminate dynamic-scheduling overhead under uniform workloads.

All commands are automated in `script.sh`, and the timing logs are saved to:

- `output.txt`: results for the large-file test
- `output2.txt`: results for the small-file test

3.1 Test results

Results of our tests report that we were able to achieve a decent speedup for both the datasets. Even though we are far from an ideal speedup, it should be noted that the problem of file compressing has unavoidable serial phases limiting the maximum speedup.

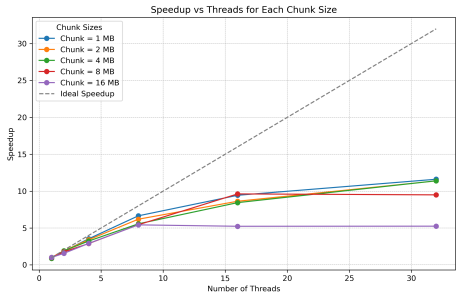

Figure 3.1: Speedup vs. Threads on the large dataset

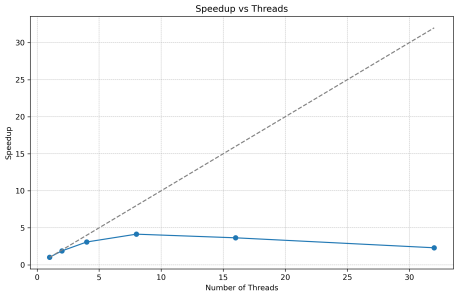

Figure 3.2: Speedup vs. Threads on the small dataset

Performance gains begin to stabilize at around 16 threads, as the overhead of thread management starts to outweigh the benefits of per-chunk parallelism. It's quite possible that even higher speedups could be achieved on larger files, but further testing would be required to confirm this.

In the small-file scenario, the peak speedup is only about 5× at 8 threads, and it falls off sharply beyond that. Because parallelism here is limited to the file level—each thread compresses a single 100 KiB file sequentially—the fixed cost of spawning and scheduling threads quickly outweighs the small amount of work per file.

Regarding chunk size, our results show that the best performance is obtained by setting the chunk size to 1 MiB, probably because in this way each thread works on roughly 3 chunks, thus achieving a balanced workload, while in the worst case, 16 MiB, some of the threads do not compress/decompress anything, causing a decrease in speedup.

It could be possible that an even smaller chunk size could grant a higher speedup, but more tests are required in order to check this.

4 Limitations and Future Work

Our parallel compressor still has room for improvements:

1. True Parallel walkdir: the current implementation of the function runs sequentially, with one thread in charge of working on all the files contained in the sub-directory. Replacing it for a version able to parallelize the work at file level even inside sub-directory would improve results in compression and decompression speed of datasets with many files and directories.

2. Use of the `compress2()` function: Miniz library function `compress2()` enables the loss of a fraction of the compression ratio for a faster execution. We decided not to use it in our implementation to show how just a few OpenMP pragmas can make a sequential code run significantly faster.

4.1 Oversubscription with Nested Parallelism

After various tests on many mid-sized files (each larger than our 1 MiB chunk threshold), we saw disappointingly low speedups despite the results shown when compressing one large file. The main cause could be that nested `#pragma omp parallel for` regions were spawning far more threads than desired.

A possible fix with our machine could be the following:

1. Limit the outer loop to 5 threads,

2. Allow each of those to spawn up to 6 threads for chunked compression,

3. Capping total concurrency at $5 \times 6 = 30$ threads, under our 32-thread limit.

In This way we preserve both file-level and chunk-level parallelism without oversaturating CPU or memory-allocator resources.